Modular EV Data Logger Dashboard Framework (MEDDF)

Enterprise Technical Architecture Document

Table of content

1. Executive Summary
2. Business Context
3. Problem Statement
4. System Vision
5. Architectural Goals
    - Configuration-Driven Architecture
    - Zero Hardcoding Principle
    - Multi-Vehicle Support
    - Real-Time Telemetry Handling
6. Non-Functional Requirements
7. Design Constraints
8. Success Criteria
9. Technology Deep Dive
    - JavaScript Runtime
    - React Internals
    - Zustand State Management

    - Axios Networking
    - WebSocket Protocol
    - Vite Build System
    - Tailwind CSS Engine

10. System Architecture

- System Context Diagram
- Layered Architecture
- Data Flow Diagram
- Component Interaction Flow
- Layer Deep Dive

11. Configuration Engine Design

- vehicleConfigs Schema
- Layout & Tile Schema
- Theme & Rule Engine
- Zod Validation
- Dashboard Rendering Algorithm
- Tile Registry (Factory Pattern)

12. Real-Time Data Engineering

- WebSocket Lifecycle
- Data Normalization Pipeline
- Selective Subscription
- Throttling & Frame Optimization

- Runtime Execution Trace

13. Performance Engineering

- Code Splitting & Lazy Loading
- Memoization Strategy
- Object Reference Stability
- Layout Optimization
- Lighthouse & Web Vitals

14. Enterprise Readiness

- Security Architecture
- Reliability Engineering
- Scalability Strategy
- DevOps & Deployment

15. Enterprise Architecture Audit

1. Executive Summary

The **Modular EV Data Logger Dashboard Framework (MEDDF)** is an enterprise-grade, configuration-driven frontend architecture designed to render dynamic EV telemetry dashboards in real time.

MEDDF enables scalable, maintainable, and extensible visualization of electric vehicle data without hardcoded dashboard logic. The framework decouples dashboard structure from implementation by leveraging configuration-based rendering, selective state subscription, and dynamic component resolution.

The system is engineered to:

- Support multiple vehicle types without code changes

- Process high-frequency telemetry streams efficiently
- Minimize re-render overhead through fine-grained state management
- Maintain strict separation between data acquisition and presentation
- Enable enterprise scalability and multi-tenant expansion

MEDDF transforms real-time telemetry data into a deterministic, dynamic, and extensible dashboard system suitable for production-grade EV platforms.

---

2. Business Context

The electric mobility ecosystem is evolving rapidly. Fleet operators, OEMs, and mobility providers require real-time observability into vehicle performance metrics such as:

- Speed
- Battery health
- Motor temperature
- GPS position
- Diagnostic indicators

Traditional dashboard systems are typically:

- Hardcoded per vehicle model
- Tightly coupled to backend schemas
- Difficult to extend
- Expensive to maintain

As product lines expand and telemetry complexity increases, organizations require:

- A scalable frontend architecture
- Rapid onboarding of new vehicle models
- Real-time telemetry visualization
- Operational reliability at scale

MEDDF addresses these enterprise needs through architectural modularity and configuration-driven composition.

---

3. Problem Statement

Existing EV dashboard implementations suffer from:

1. Hardcoded UI logic per vehicle
2. Tight coupling between backend schema and frontend rendering
3. Code duplication across models
4. Performance degradation under high-frequency telemetry
5. Poor maintainability and slow iteration cycles

When introducing a new vehicle model:

- Developers must write new components
- Modify layout logic
- Adjust data mapping
- Redeploy the application

This approach does not scale across:

- 10+ vehicle models

- Multi-tenant deployments
- High telemetry throughput environments

A new architectural approach is required—one that abstracts structure from implementation and enables dynamic runtime composition.

---

4. System Vision

The vision of MEDDF is to establish a **configuration-driven, real-time dashboard engine** capable of rendering multiple vehicle dashboards dynamically without modifying core engine code.

The system must:

- Treat dashboard structure as data (JSON configuration)
- Treat telemetry as a real-time event stream
- Render UI dynamically based on configuration
- Update only affected components during state changes
- Support extensibility through plugin-style tile registration

MEDDF is not a single dashboard.
It is a **dashboard engine framework**.

---

5 Architectural Goals

5.1 Configuration-Driven Architecture

Why Configuration-Driven?

Configuration-driven architecture allows:

- Separation of structure from implementation
- Dynamic runtime composition
- Zero redeployment for new vehicle onboarding
- Rapid experimentation and customization

Instead of:

If vehicleA → Render DashboardA
If vehicleB → Render DashboardB

We implement:

Load vehicleConfig → Render dynamically

This design ensures horizontal scalability and future-proof extensibility.

5.2 Zero Hardcoding Principle

Hardcoded dashboards introduce:

- Maintenance burden
- Duplication
- Tight coupling
- Scalability limitations

MEDDF enforces:

- No vehicle-specific rendering logic
- No conditional UI branches per model
- No static layout definitions in components

All structure is externalized into configuration files.

This adheres to:

- Open-Closed Principle
- Single Responsibility Principle
- Dependency Inversion Principle

5.3 Multi-Vehicle Support

The architecture must support:

- 5+ vehicle models initially
- 20+ models in scaling phase
- Tenant-based dashboard variations

Multi-vehicle support is achieved by:

- Vehicle detection module
- Per-vehicle configuration schemas
- Dynamic component resolution
- Unified rendering engine

Vehicle onboarding process becomes configuration-only.

5.4 Real-Time Telemetry Handling

Real-time telemetry is critical because:

- EV systems generate frequent state updates

- Operational decisions depend on low-latency visibility
- Fleet monitoring requires live accuracy

System must handle:

- High-frequency WebSocket streams
- Burst telemetry updates
- Partial data frames
- Network instability

Performance goal:

- Maintain <16ms render budget
- Avoid full dashboard re-render
- Use selective state subscription

Real-time capability is foundational—not optional.

---

6. Non-Functional Requirements

6.1 Performance

- Initial load < 3 seconds
- Frame render time < 16ms
- Minimal garbage collection pressure
- O(1) subscription granularity

- O(n) tile render complexity

6.2 Scalability

- Support 20+ vehicle configurations
- Extensible tile registry
- Plugin-ready architecture
- Micro-frontend compatible

6.3 Reliability

- WebSocket reconnection with backoff
- Polling fallback strategy
- Circuit breaker logic
- Error boundary protection

6.4 Security

- Token-based API access
- Configuration validation
- XSS prevention
- Network boundary isolation

6.5 Maintainability

- Layered architecture
- Strict dependency direction
- Modular folder structure
- Clear separation of concerns

---

7. Design Constraints

7.1 Backend Independence

The frontend engine must:

- Treat backend as external dependency
- Not assume database schema control
- Normalize incoming payloads

7.2 Browser Environment Limitations

- JavaScript single-threaded runtime
- Memory constraints
- Network variability
- Event-loop scheduling

7.3 Rendering Budget Constraint

UI must remain interactive even during high telemetry bursts.

Hard constraint:

> Render + commit + paint must not exceed frame budget.

7.4 Deployment Constraints

- Must deploy as static asset bundle
- CDN-compatible
- Containerizable
- Environment-configurable

---

8. Success Criteria

MEDDF will be considered successful when:

1. New vehicle dashboard can be added via configuration only.
2. No engine code changes required for new vehicle model.
3. Real-time updates do not trigger full dashboard re-render.
4. System supports 20+ vehicle configs without architectural change.
5. Telemetry latency from WebSocket to paint < 50ms.
6. Production build passes Lighthouse performance benchmarks.
7. System remains stable under simulated telemetry bursts.

🔷 Strategic Justification Summary

Design Decision	Justification
Configuration-driven	Enables dynamic runtime composition
Zero hardcoding	Reduces maintenance & improves scalability
Multi-vehicle support	Required for EV product line expansion
Real-time telemetry	Critical for operational monitoring
Selective state management	Ensures performance stability
Dynamic tile registry	Enables extensibility

Closing Statement

MEDDF is not merely a UI application; it is a scalable dashboard framework engineered for enterprise EV ecosystems.

It embodies:

- Architectural modularity
- Runtime efficiency
- Scalable extensibility
- Operational reliability
- Production readiness

This framework establishes a long-term foundation for dynamic telemetry visualization across evolving EV platforms.

9. MEDDF – Technology Deep Dive

Audience: Principal Engineers / CTO / Runtime Architects
Scope: Internal mechanics of the core technology stack powering MEDDF

This section analyzes the internal behavior of each technology at runtime, build-time, and memory level. This is not usage documentation. This is systems-level explanation.

---

1. JavaScript – Runtime & Execution Semantics

MEDDF executes entirely inside the browser's JavaScript runtime (V8 / SpiderMonkey / WebKit JS engine). Understanding the execution model is essential for real-time dashboard stability.

1.1 ES6 Modules (ESM)

Internal Mechanics

ES Modules are **statically analyzable**. During parsing:

1. The module graph is constructed.
2. Imports are resolved before execution.
3. Bindings are created as live references (not copies).

```
import { useVehicleStore } from './store'
```

Internally:

- Module has its own scope.
- Exports are placed in a module namespace object.
- Imports reference memory location directly.

Why It Matters

- Enables tree shaking (unused exports removed).
- Prevents runtime `require()` resolution.
- Guarantees deterministic dependency graph.

1.2 Arrow Functions

Arrow functions:

const update = (data) => set(data)

Internal behavior:

- No `this` binding.
- Lexically captures outer scope.
- Cannot be used as constructors.
- No `arguments` object.

Used heavily in selectors and state updates to avoid context rebinding overhead.

1.3 Promises & Async/Await

Promise Lifecycle

States:

PENDING → FULFILLED → REJECTED

Internally:

- Promise stored in heap.
- Callback functions queued in microtask queue.

Async/Await compiles to:

```
function asyncFunc() {
 return new Promise(...)
}
```

`await` pauses execution, pushes continuation to microtask queue.

1.4 Event Loop

JavaScript runtime model:

```
Call Stack
↓
Microtask Queue (Promises)
↓
Macrotask Queue (setTimeout, I/O)
```

Execution order:

1. Execute call stack
2. Drain microtask queue
3. Render
4. Next macrotask

In MEDDF:

WebSocket → onmessage → setState → microtask scheduling → React render

---

1.5 Call Stack

When WebSocket frame arrives:

```
onmessage()
 → JSON.parse()
 → normalize()
 → store.set()
```

→ React scheduleUpdate()

If call stack grows too deep → frame drops possible.

1.6 Memory Model

JavaScript memory:

- Stack: primitives & execution context
- Heap: objects, closures, promises

Telemetry objects allocated on heap.

Garbage collection:

- Mark-and-sweep
- Generational GC

Short-lived telemetry objects allocated in young generation → fast cleanup.

1.7 Closures

Closures capture lexical scope.

Example:

```
function createRetry() {
 let attempt = 0
 return () => attempt++
}
```

Captured variables remain in heap until reference dropped.

Memory leaks possible if event listeners not cleaned.

2.React – Rendering Engine Internals

2.1 Virtual DOM

React creates a lightweight tree representation.

On update:

1. New tree generated
2. Diffed against previous
3. Minimal DOM mutations applied

Avoids full DOM rebuild.

2.2 Reconciliation Algorithm

React compares:

- Element type
- Key
- Props

Heuristic O(n) diffing.

Stable `key` prop critical for tile preservation.

2.3 Fiber Architecture

Fiber = linked list tree.

Enables:

- Interruptible rendering
- Priority scheduling
- Time slicing

Fiber nodes contain:

- memoizedState
- updateQueue
- child / sibling pointers

This enables concurrent rendering.

2.4 Rendering Lifecycle

Two phases:

Render Phase

- Compute virtual tree
- Pure
- Interruptible

Commit Phase

- Apply DOM mutations
- Run useEffect
- Non-interruptible

Telemetry updates occur in render phase.

2.5 Hooks Internals

Hooks stored in linked list attached to fiber.

Each render:

- Hook index increments
- State stored in fiber.memoizedState

Order-sensitive.

2.6 useEffect Timing

Runs after commit phase.

Sequence:

Render → Commit → Paint → useEffect

Used for:

- WebSocket setup
- Cleanup logic

2.7 Batching Updates

React 18 batches async updates automatically.

If multiple setState calls occur in same tick:

- Single render pass executed.

Reduces render frequency.

2.8 React.memo

Higher-order component.

Stores previous props.
Shallow compares new props.

Skips render if equal.

2.9 useMemo

Caches computed value.

const value = useMemo(() => expensiveCalc(), [deps])

Avoid recomputation during unrelated updates.

2.10 useCallback

Memoizes function reference.

Prevents child re-render due to new function identity.

---

3. Zustand – State Management Internals

3.1 Store Creation

create((set, get) => ({ state }))

Internally:

- State object
- Listeners array

- set() method
- subscribe() method

3.2 Subscription Model

Each component registers selector.

On state update:

1. Selector executed
2. Compared to previous result
3. If changed → component re-render

Selective granularity.

3.3 Shallow Comparison

Compares keys shallowly.

Avoids deep object traversal.

Prevents unnecessary re-render.

3.4 Avoiding Global Re-render

Never subscribe to full state.

Always subscribe to specific property:

useStore(state => state.telemetry.speed)

---

4. Axios – HTTP Client Internals

4.1 Request Lifecycle

1. Merge config
2. Run request interceptors
3. Execute adapter (XHR)
4. Run response interceptors
5. Return Promise

Interceptor chain behaves like middleware.

4.2 Error Handling

Errors propagate via rejected Promise.

Centralized handling through response interceptor.

5.WebSocket – Protocol-Level Mechanics

5.1 TCP Handshake

HTTP upgrade → persistent TCP connection.

No repeated headers.

Low latency.

5.2 Frame Structure

Frame includes:

- FIN bit
- Opcode
- Mask
- Payload length
- Payload

Binary framing reduces overhead.

5.3 Lifecycle

CONNECT
→ onopen
→ onmessage*
→ onerror?
→ onclose

5.4 Reconnection Strategy

Exponential backoff + jitter.

Prevents server overload.

5.5 Heartbeat Mechanism

Ping/Pong to detect dead connections.

```
setInterval(() => socket.send("ping"), 30000)
```

---

6. Vite – Build System Internals

6.1 ESM Dev Server

Serves modules directly.

No bundling in dev.

Fast reload.

6.2 Rollup Bundling

Production:

- Static analysis
- Bundle graph
- Code splitting
- Minification

6.3 HMR

Module replacement without full reload.

Preserves component state.

6.4 Tree Shaking

Unused exports removed.

Reduces bundle size.

---

7. Tailwind CSS – Styling Engine

7.1 Utility-First CSS

Each class = atomic rule.

Example:

Minimal CSS duplication.

7.2 JIT Engine

Scans project files.

Generates only used classes.

Reduces CSS size drastically.

7.3 Responsive System

Mobile-first breakpoints:

sm: md: lg: xl:

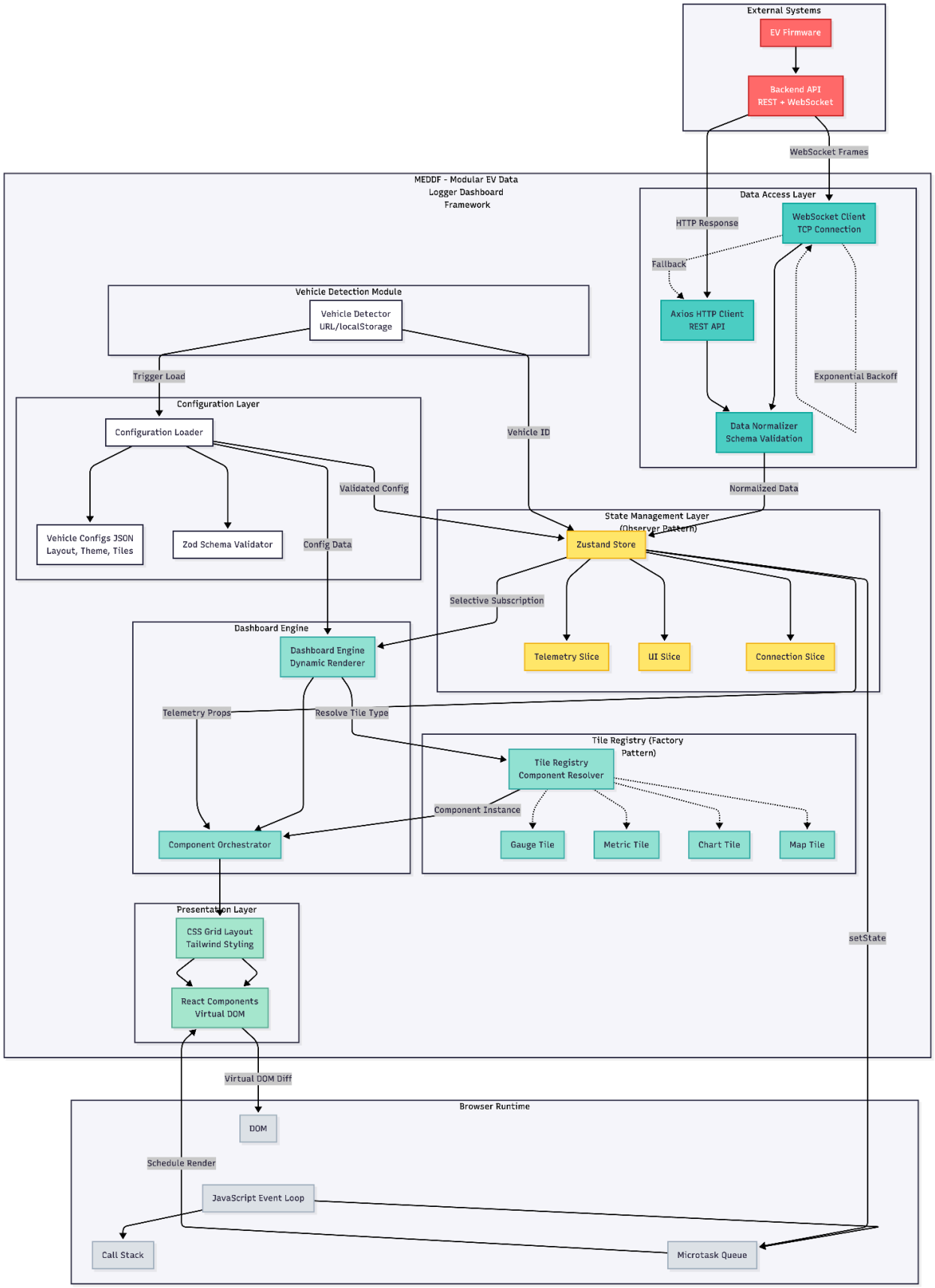
Compiled to media queries.

10.Modular EV Data Logger Dashboard Framework (MEDDF)

Complete System Architecture – Enterprise Specification

Audience: CTO / Principal Engineers / Senior Architects
Scope: End-to-End Frontend System Architecture

This document defines the complete system architecture of MEDDF including context, layering, runtime behavior, dependency flow, memory implications, and applied design patterns.


External Systems
EV Firmware
Backend API
REST + WebSocket
WebSocket Frames
MEDDF - Modular EV Data
Logger Dashboard
Framework
Data Access Layer
HTTP Response
WebSocket Client
TCP Connection
Fallback
Axios HTTP Client
REST API
Exponential Backoff
Data Normalizer
Schema Validation
Normalized Data
Vehicle Detection Module
Vehicle Detector
URL/localStorage
Trigger Load
Vehicle ID
Configuration Layer
Configuration Loader
Validated Config
Vehicle Configs JSON
Layout, Theme, Tiles
Zod Schema Validator
Config Data
State Management Layer
(Observer Pattern)
Zustand Store
Selective Subscription
Telemetry Slice
UI Slice
Connection Slice
Dashboard Engine
Dashboard Engine
Dynamic Renderer
Telemetry Props
Resolve Tile Type
Tile Registry (Factory
Pattern)
Tile Registry
Component Resolver
Component Instance
Gauge Tile
Metric Tile
Chart Tile
Map Tile
Component Orchestrator
Presentation Layer
CSS Grid Layout
Tailwind Styling
React Components
Virtual DOM
setState
Virtual DOM Diff
Browser Runtime
DOM
Schedule Render
JavaScript Event Loop
Call Stack
Microtask Queue

1. System Context Diagram

MEDDF operates within a broader EV telemetry ecosystem.

```
+-----------------------------------------------------+
|                External Systems                     |
|-----------------------------------------------------|
|  EV Firmware  →  Backend API  →  WebSocket Server   |
+-----------------------------------------------------+
                    |
                    |  (REST + WebSocket)
                    ▼
+-----------------------------------------------------+
|     Modular EV Data Logger Dashboard Framework      |
|                  (Frontend)                         |
+-----------------------------------------------------+
                    |
                    ▼
                End User (Browser)
```

Context Explanation

- EV firmware generates telemetry data.
- Backend exposes REST endpoints and WebSocket streams.
- MEDDF consumes backend data but does not own backend.
- Browser is the execution environment.

System boundary: MEDDF begins at the browser runtime and treats backend as an external dependency.

---

2. Layered Architecture

MEDDF follows strict layered architecture with unidirectional dependency flow.

Presentation Layer
▲
Dashboard Engine
▲
Tile Registry (Factory)
▲
Configuration Layer
▲
Vehicle Detection Module
▲
State Management Layer (Observer)
▲
Data Access Layer
▲
External Backend

Dependency Direction

- Upper layers depend only on lower abstractions.
- No upward references allowed.
- Store never imports UI.
- Tiles never import data access.
- Data access never imports rendering logic.

Prevents circular dependencies.

---

3. Data Flow Diagram

WebSocket Message
│
▼
Data Access Layer
│ (Normalize)
▼
State Management Layer (Zustand)
│ (Selective subscription)

```
    ▼
Dashboard Engine
    │
    ▼
Tile Registry
    │
    ▼
Tile Component
    │
    ▼
DOM Update
```

Execution Flow

1. Telemetry received
2. Payload normalized
3. Immutable store update
4. Observer triggers selective re-render
5. Virtual DOM diff
6. Commit phase patch
7. Browser paint

---

4. Component Interaction Flow

```
App.jsx
↓
Vehicle Detection
↓
Load Configuration
↓
Initialize Store
↓
DashboardEngine
↓
```

TileRegistry.resolve()
↓
Render Tile Components
↓
Subscribe to Store

Interaction model is event-driven and deterministic.

Layer Deep Dive

🔵 Data Access Layer

Responsibility

- REST API calls
- WebSocket connection
- Network boundary isolation
- Payload normalization
- Error handling

Dependency Direction

- Depends on Axios & WebSocket.
- Does not depend on Store.
- Exposes normalized data to Store.

Runtime Execution Order

1. Initialize API client
2. Establish WebSocket
3. Receive message
4. Normalize data
5. Pass to Store

Memory Considerations

- Temporary objects created during normalization.
- Shallow copies used.
- No deep object nesting.

Circular Dependency Prevention

- Store never imports API client.
- Data layer exports pure functions only.

🔵 State Management Layer (Observer Pattern)

Responsibility

- Central runtime state container.
- Telemetry slice.
- UI slice.

- Connection slice.

Observer Pattern Implementation

Store maintains listener registry.

Pseudo:

```
state = {}
listeners = []

function subscribe(listener) {
 listeners.push(listener)
}

function set(newState) {
 state = { ...state, ...newState }
 listeners.forEach(listener => listener())
}
```

Selector-based subscription prevents global re-render.

Runtime Execution Order

1. set() invoked
2. State shallow cloned
3. Listeners notified
4. Selector executed
5. Compare previous value
6. If changed → React render scheduled

Memory Considerations

- Telemetry object shallow cloned.

- Old object marked for GC.
- Flat structure reduces GC traversal cost.

Circular Dependency Prevention

- Store unaware of UI.
- No import from components.

🔵 Vehicle Detection Module

Responsibility

- Determine active vehicle.
- Resolve configuration context.

Execution Order

1. Parse URL
2. Check localStorage
3. Fallback to default
4. Set store.vehicleId

Memory Consideration

Minimal; primitive values only.

Dependency Direction

- Depends on browser API.
- Writes to Store.
- Does not import DashboardEngine.

🔵 Configuration Layer

Responsibility

- Define layout.
- Define tile types.
- Define data mapping.

Structure

```
{
 layout: {...},
 theme: "...",
 tiles: [...]
}
```

Runtime Execution Order

1. Load config JSON
2. Validate schema
3. Store config in state
4. DashboardEngine consumes config

Memory Consideration

- Loaded once per vehicle switch.
- Remains stable reference.

Circular Dependency Prevention

- Config does not import store.
- Pure data object.

🔵 Dashboard Engine

Responsibility

- Dynamic component orchestration.
- Reads config.
- Resolves tile components.
- Injects props.

Runtime Algorithm

```
for each tile in config.tiles:
  component = TileRegistry[tile.type]
  render(component, props)
```

Time Complexity: O(n) per render where n = number of tiles.

Memory Considerations

- JSX generates virtual nodes.
- Reconciliation prevents full DOM rebuild.

Dependency Direction

- Depends on Store.
- Depends on TileRegistry.
- Does not depend on Data Access.

🔵 Tile Registry (Factory Pattern)

Factory Pattern Implementation

Registry maps string → component.

```
const TileRegistry = {
 gauge: GaugeTile,
 chart: ChartTile
}
```

Engine does not know concrete classes.

Why Factory Pattern

- Eliminates conditional branching.
- Enables extension without modifying engine.
- Supports plugin model.

Memory Consideration

Registry static reference; minimal overhead.

🔵 Presentation Layer

Responsibility

- Layout rendering.
- Tailwind styling.
- Responsive design.

Runtime Execution

1. React render phase builds virtual tree.
2. Reconciliation diff.
3. Commit DOM patches.
4. CSS layout recalculation.
5. Paint.

Performance

- CSS Grid avoids manual layout calculation.
- Utility classes minimize style recalculation.

Runtime Execution Order (Full)

1. App Mount
2. Vehicle Detection
3. Config Load
4. Store Initialization
5. REST Fetch

6. WebSocket Connect
7. Dashboard Render
8. Real-time Update Loop

Memory Allocation Model

Telemetry Update:

- Allocate new shallow object
- Replace reference
- GC old object

Tile Render:

- Virtual node allocated
- Reused via reconciliation if key stable

Avoid retained closures to prevent leaks.

Circular Dependency Prevention Strategy

Rules enforced:

- Data layer cannot import UI.
- Store cannot import rendering layer.
- TileRegistry does not depend on Store.
- Config is pure data.

Dependency graph is strictly acyclic.

11.–MEDFF Configuration Engine Technical Design

Audience: Principal Engineers / System Architects
Scope: Deep technical specification of the Configuration-Driven Rendering Engine

The Configuration Engine is the structural core of MEDDF.
It enables dynamic dashboard composition without hardcoded logic.

This document defines:

- Configuration schema design
- Validation strategy
- Layout computation
- Rendering algorithm
- Extensibility model

1.Configuration Philosophy

The Configuration Engine follows:

- **Structure as Data**
- **Zero Hardcoding**
- **Open-Closed Principle**
- **Runtime Composability**

The engine never contains vehicle-specific logic.
All structure is externalized into JSON configuration.

---

2. vehicleConfigs.json Schema

Each vehicle configuration defines:

- Metadata
- Layout system
- Theme configuration
- Tile composition
- Optional rules

Example vehicleConfigs.json

```
{
 "vehicleA": {
   "meta": {
     "name": "Scooter 703-O",
     "version": "1.0"
   },
   "layout": {
     "columns": 12,
     "rowHeight": 100,
     "gap": 16
   },
   "theme": {
     "mode": "dark",
     "primaryColor": "#00E5FF",
     "background": "bg-gray-900"
   },
   "tiles": [
     {
       "id": "speedTile",
       "type": "gauge",
       "dataKey": "speed",
       "position": { "x": 0, "y": 0, "w": 4, "h": 2 }
```

```
    },
    {
      "id": "batteryTile",
      "type": "metric",
      "dataKey": "battery",
      "position": { "x": 4, "y": 0, "w": 4, "h": 2 }
    }
  ]
 }
}
```

3.Layout Schema

Layout defines grid system parameters.

```
interface LayoutSchema {
 columns: number
 rowHeight: number
 gap: number
}
```

Grid Layout Computation

Given:

```
{ "x": 2, "y": 1, "w": 4, "h": 2 }
```

CSS Grid Computation:

```
const gridStyle = {
 gridColumn: `${x + 1} / span ${w}`,
 gridRow: `${y + 1} / span ${h}`
}
```

Grid Math

If 12-column system:

Column width = containerWidth / 12

Start column = x + 1
End column = start + w

Time Complexity: O(n) per tile mapping.

---

4. Tile Schema

```
interface TileSchema {
 id: string
 type: string
 dataKey: string
 position: {
   x: number
   y: number
   w: number
   h: number
 }
 props?: Record<string, any>
}
```

Fields:

- id → Stable React key
- type → Factory resolution
- dataKey → Store mapping
- position → Layout math
- props → Component customization

---

5. Theme Schema

```
interface ThemeSchema {
 mode: "light" | "dark"
 primaryColor: string
 background: string
```

```
}
```

Theme injected via state:

```
const theme = useStore(state => state.ui.theme)
```

Tailwind classes applied dynamically.

---

6. Rule Engine (Optional Extension)

Rules allow dynamic behavior based on telemetry.

Example:

```
{
 "rules": [
   {
     "condition": "battery < 20",
     "action": "highlight",
     "target": "batteryTile"
   }
 ]
}
```

Rule Engine Pseudo Code

```
function evaluateRules(rules, telemetry) {
 for (rule of rules) {
   if (evalCondition(rule.condition, telemetry)) {
     applyAction(rule)
   }
 }
}
```

Safer implementation uses expression parser instead of eval.

---

7. Configuration Validation Using Zod

Zod ensures runtime schema safety.

Zod Schema

```
import { z } from "zod"

const PositionSchema = z.object({
 x: z.number(),
 y: z.number(),
 w: z.number(),
 h: z.number()
})

const TileSchema = z.object({
 id: z.string(),
 type: z.string(),
 dataKey: z.string(),
 position: PositionSchema,
 props: z.optional(z.record(z.any()))
})

const LayoutSchema = z.object({
 columns: z.number(),
 rowHeight: z.number(),
 gap: z.number()
})

const ThemeSchema = z.object({
 mode: z.enum(["light", "dark"]),
 primaryColor: z.string(),
 background: z.string()
})

const VehicleConfigSchema = z.object({
 meta: z.object({
   name: z.string(),
   version: z.string()
 }),
 layout: LayoutSchema,
 theme: ThemeSchema,
 tiles: z.array(TileSchema)
})
```

Validation Usage

const parsed = VehicleConfigSchema.parse(config)

If invalid → throw error before rendering.

Prevents runtime crashes.

---

8. Dynamic Tile Rendering Algorithm

Core logic of DashboardEngine:

Load Config
↓
Validate Schema
↓
Iterate Tiles
↓
Resolve Component via Registry
↓
Inject Props
↓
Render JSX

---

9. DashboardEngine Pseudo Code

```
function DashboardEngine() {
 const vehicleId = useStore(state => state.vehicle.id)
 const telemetry = useStore(state => state.telemetry)
 const config = vehicleConfigs[vehicleId]

 validateConfig(config)

 return (
   <div
     className="grid"
     style={{
```

```
      gridTemplateColumns: `repeat(${config.layout.columns}, 1fr)`,
      gap: config.layout.gap
    }}
  >
    {config.tiles.map(tile => {
      const Component = TileRegistry[tile.type]

      if (!Component) {
        console.warn("Unknown tile type:", tile.type)
        return null
      }

      const value = telemetry[tile.dataKey]

      return (
        <Component
          key={tile.id}
          value={value}
          position={tile.position}
          {...tile.props}
        />
      )
    })}
  </div>
 )
}
```

---

10. Dynamic Tile Resolution (Factory Pattern)

TileRegistry:

```
const TileRegistry = {
 gauge: GaugeTile,
 metric: MetricTile,
 chart: ChartTile,
 map: MapTile
}
```

Engine never uses switch-case.

New tile addition:

TileRegistry["heatmap"] = HeatmapTile

No engine modification required.

---

11. Adding New Vehicle (Zero Code Change)

Steps:

1. Add new vehicle config in vehicleConfigs.json:

"vehicleB": { ... }

2. Ensure telemetry keys match dataKey.
3. Load via URL:

/?vehicle=vehicleB

Engine automatically:

- Detects vehicle
- Loads config
- Renders tiles

No code change.

No redeployment logic modification.

---

12. Dependency Direction

DashboardEngine → TileRegistry
DashboardEngine → Store
Store ← DataAccessLayer
Config → Engine

Config is pure data.

TileRegistry independent of Store.

Acyclic graph.

---

13. Memory Considerations

- Config loaded once per vehicle.
- Tiles array reused.
- Telemetry shallow cloned.
- Stable key ensures fiber reuse.
- No deep nested objects.”

12.MEDFF– Real-Time Data Engineering

Audience: Principal Engineers / Performance Architects
Scope: Real-time execution pipeline, performance constraints, and render optimization

This section defines how MEDDF transforms high-frequency telemetry streams into deterministic UI updates while maintaining a <16ms frame budget.

---

1. WebSocket Lifecycle – Low-Level Runtime Mechanics

MEDDF uses the native browser WebSocket API built over TCP.

1.1 TCP Upgrade Handshake

WebSocket begins as HTTP:

```
GET /ws HTTP/1.1
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: <random>
```

Server replies:

```
HTTP/1.1 101 Switching Protocols
```

After upgrade:

- Persistent TCP channel
- Full-duplex communication
- Frame-based transmission
- No repeated HTTP headers

This reduces latency significantly compared to polling.

1.2 WebSocket Connection Lifecycle

```
CONNECT
→ onopen
→ onmessage*
→ onerror?
→ onclose
```

onopen

```
socket.onopen = () => {
 setConnectionStatus("CONNECTED")
}
```

Executed when TCP handshake completes.

---

onmessage

```
socket.onmessage = (event) => {
 const raw = JSON.parse(event.data)
 const normalized = normalizeTelemetry(raw)
 updateStore(normalized)
}
```

Call stack trace:

```
onmessage
 → JSON.parse
 → normalizeTelemetry
 → store.set
 → React scheduleUpdate
```

---

onerror

Triggered on network failure.
Does not guarantee closure.

---

onclose

Receives close code:

- 1000 → normal
- 1006 → abnormal termination

Triggers reconnection logic.

---

2. Exponential Backoff Reconnection

Avoid infinite retry storms.

Algorithm:

```
delay = min(baseDelay * 2^attempt, MAX_DELAY)
```

Pseudo-code:

```
let attempt = 0


function reconnect() {
 const delay = Math.min(1000 * 2 ** attempt++, 30000)
 setTimeout(connectWebSocket, delay)
}
```

Add jitter:

```
delay += Math.random() * 500
```

Prevents synchronized reconnect bursts across clients.

---

3. Data Normalization Pipeline

Telemetry from backend is untrusted and unstable.

Pipeline:

```
Raw Frame
 → JSON.parse
 → Schema validation
 → Flatten nested structure
 → Sanitize values
 → Timestamp injection
 → Immutable state update
```

Example

Incoming:

```
{
 "spd": 75,
 "bat_lvl": 88,
 "motor": { "temp": 42 }
}
```

Normalization:

```
function normalizeTelemetry(raw) {
 return {
   speed: raw.spd ?? 0,
   battery: raw.bat_lvl ?? 0,
   motorTemp: raw.motor?.temp ?? 0,
   _ts: performance.now()
 }
```

```
}
```

---

4. Timestamp Management

Each telemetry update is timestamped:

```
_ts: performance.now()
```

Uses high-resolution clock.

Purpose:

- Detect stale updates
- Calculate latency
- Monitor update frequency

Optional drift correction logic:

```
if (now - lastUpdate > threshold) markStale()
```

---

5. Zustand Selective Subscription

Zustand implements Observer pattern.

Store Structure

```
{
 telemetry: { speed, battery, temp },
 connection: { status }
}
```

Subscription Example

```
const speed = useStore(state => state.telemetry.speed)
```

Update sequence:

```
store.set()
 → iterate listeners
 → execute selector
 → shallow compare
 → if changed → schedule re-render
```

Critical: Avoid subscribing to entire telemetry object.

Bad:

```
useStore(state => state.telemetry)
```

Triggers global re-render.

6.Throttling Telemetry Updates

If WebSocket sends 60+ updates/sec, rendering every frame is unnecessary.

Strategy 1 – Time-Based Throttle

```
let lastUpdate = 0


function handleTelemetry(data) {
 const now = performance.now()
```

```
 if (now - lastUpdate > 100) {
   store.set(data)
   lastUpdate = now
 }
}
```

Limits to 10 updates/sec.

Strategy 2 – requestAnimationFrame Alignment

```
let pendingData = null

socket.onmessage = (event) => {
 pendingData = JSON.parse(event.data)
}

function processFrame() {
 if (pendingData) {
   store.set(normalizeTelemetry(pendingData))
   pendingData = null
 }
 requestAnimationFrame(processFrame)
}
```

Aligns updates to paint cycle.

---

7. Preventing Unnecessary Re-renders

Granular Subscription

Each tile subscribes only to required key.

React.memo

```
export default React.memo(GaugeTile)
```

Prevents re-render when props unchanged.

Stable Object References

Immutable update pattern:

```
set(state => ({
 telemetry: {
   ...state.telemetry,
   speed: newSpeed
 }
}))
```

Only `speed` reference changes.

---

8. <16ms Render Optimization

60 FPS requires:

1000ms / 60 = 16.67ms per frame

Total time includes:

- JS execution
- React render
- DOM patch
- Layout
- Paint

Optimization Strategies

1. Flat telemetry structure
2. Shallow clones only
3. Stable key props
4. Avoid layout thrashing
5. Use CSS Grid
6. Minimize heavy computations inside render

---

9. React Render Cycle Timeline

When telemetry update occurs:

T0 WebSocket frame received

T1 onmessage callback enters call stack

T2 JSON.parse (~0.1ms)

T3 normalizeTelemetry (~0.1ms)

T4 store.set (~0.05ms)

T5 React schedules update

T6 Render phase (diff virtual DOM)

T7 Commit phase (DOM patch)

T8 Browser layout & paint

Render phase is interruptible.
Commit phase is atomic.

---

10. Runtime Execution Trace Example

Scenario: Speed changes from 70 → 75.

Step-by-Step Trace

1. TCP frame received.
2. onmessage fires.
3. JSON.parse executes.
4. normalizeTelemetry returns `{ speed: 75 }`.
5. store.set called.
6. Zustand notifies subscribers.
7. Only speedTile selector detects change.
8. React schedules render for GaugeTile.
9. Virtual DOM diff sees only text node change.

10. DOM textContent updated.

11. Browser paints updated value.

Other tiles remain untouched.

---

11. End-to-End Real-Time Flow

WebSocket Frame

↓

Event Loop

↓

Call Stack

↓

Normalization

↓

Immutable Store Update

↓

Observer Pattern (Selective Notify)

↓

React Render Phase

↓

Reconciliation

↓

Commit Phase

↓

Paint

Deterministic. Bounded. Scalable.

13. MEDDF – Performance Engineering

Audience: CTO / Principal Engineers / Performance Architects
Scope: Build-time optimization, runtime optimization, rendering efficiency, and telemetry stress handling

Performance engineering in MEDDF is treated as a first-class architectural requirement, not an afterthought. The framework is designed to support high-frequency telemetry updates while maintaining UI responsiveness and sub-16ms render targets.

---

1. Lazy Loading (React.lazy)

Internal Mechanics

`React.lazy()` enables **component-level code splitting** using dynamic imports.

```
const ChartTile = React.lazy(() => import('./ChartTile'))
```

What Happens Internally

1. `import()` returns a Promise.
2. React suspends rendering of that component.
3. Once module loads, Fiber resumes rendering.
4. Component code is added to execution context.

Why It Matters

- Reduces initial bundle size.

- Defers loading heavy components (charts, maps).
- Improves First Contentful Paint (FCP).

---

2. Code Splitting

Concept

Instead of one monolithic bundle:

app.js (2MB)

We split into:

main.js
dashboard.chunk.js
chart.chunk.js
map.chunk.js

Types of Splitting

Route-Based Splitting

```
const Dashboard = lazy(() => import('./Dashboard'))
```

Component-Based Splitting

Heavy tiles loaded only if present in config.

Benefit

- Lower Time To Interactive (TTI)
- Faster initial load
- Better Lighthouse score

---

3. Dynamic import()

Dynamic import returns a Promise:

import('./HeatmapTile').then(module => ...)

Internal Behavior:

- Browser fetches JS chunk.
- Parses & executes.
- Registers module in module registry.
- Promise resolves.

This is runtime ESM resolution.

---

4. Bundle Analysis

Using Vite:

vite build --analyze

This produces:

- Chunk size breakdown
- Duplicate dependency detection
- Tree shaking visibility

Goals

- Main bundle < 300KB gzip
- Avoid multiple charting libraries

- Eliminate unused utility functions

---

5. Tree Shaking

Tree shaking removes unused exports at build time.

Example:

```
export function heavyFunction() {}
export function usedFunction() {}
```

If only `usedFunction` imported:

`heavyFunction` removed during Rollup bundling.

Why It Works

- ESM static analysis.
- No dynamic require.
- Side-effect detection.

---

6. Memoization Strategies

Memoization prevents unnecessary recalculation and re-render.

React.memo

Prevents component re-render if props unchanged.

```
export default React.memo(GaugeTile)
```

Shallow compares props.

---

useMemo

Caches computed values:

```
const computed = useMemo(() => calculate(data), [data])
```

Avoids expensive recalculation.

---

useCallback

Prevents function identity changes:

```
const handleClick = useCallback(() => {}, [])
```

Prevents child component re-render.

---

7. Object Reference Stability

React and Zustand rely on reference comparison.

Bad:

```
telemetry = newObject
```

Good:

```
telemetry = {
 ...telemetry,
 speed: newSpeed
}
```

Only `speed` reference changes.

This ensures:

- Minimal re-render

- Selector precision
- Efficient reconciliation

---

8. Avoiding Layout Thrashing

Layout thrashing occurs when reading and writing DOM repeatedly.

Bad pattern:

```
element.offsetHeight
element.style.width = ...
```

Repeatedly forces reflow.

MEDDF Strategy

- Use CSS Grid.
- Avoid manual layout measurement.
- Avoid DOM mutation inside loops.
- Keep layout purely declarative.

---

9. High-Frequency Telemetry Optimization

When telemetry frequency > 30 updates/sec:

Problems:

- Excessive store updates.

- React render backlog.
- Dropped frames.

Strategy 1 – Throttling

```
if (now - lastUpdate > 100) {
 store.set(data)
}
```

Limits to 10 FPS.

Strategy 2 – Frame Alignment

Use `requestAnimationFrame`:

```
requestAnimationFrame(() => {
 store.set(pendingData)
})
```

Aligns with browser paint cycle.

Strategy 3 – Batch Updates

If multiple parameters change:

```
set(state => ({
 telemetry: {
   ...state.telemetry,
   ...updatedValues
 }
}))
```

Single render instead of multiple.

10.Lighthouse Metrics

Lighthouse evaluates:

- Performance
- Accessibility
- Best practices
- SEO

Performance metrics include:

- FCP
- LCP
- CLS
- TTI

Target:

Performance score ≥ 90

11.Web Vitals

FCP – First Contentful Paint

Time until first visible content.

Optimized by:

- Code splitting

- Lazy loading
- Minimal blocking JS

LCP – Largest Contentful Paint

Time until largest element rendered.

Optimize by:

- Avoiding large blocking images
- Reducing initial JS

CLS – Cumulative Layout Shift

Layout stability metric.

Prevent by:

- Fixed grid structure
- Defined tile dimensions
- Avoid dynamic height shifts

TTI – Time To Interactive

Time until page responsive.

Improve by:

- Smaller bundles

- Avoid heavy synchronous tasks
- Defer non-critical work

Runtime Performance Timeline (Telemetry Update)

WebSocket Frame
↓
JSON.parse
↓
Normalize
↓
store.set
↓
Selector Compare
↓
React Render Phase
↓
Commit Phase
↓
Paint

Goal: Entire pipeline < 16ms.

14. MEDDF – Enterprise Readiness Specification

Audience: CTO / Principal Engineers / Platform Architects
Scope: Security, Reliability, Scalability, and DevOps Hardening

This section defines how MEDDF transitions from a high-performance frontend framework into a production-ready, enterprise-grade platform capable of operating securely and reliably in large-scale EV ecosystems.

---

1. Security Architecture

Security in MEDDF is based on **zero-trust frontend assumptions**.
All network boundaries are treated as hostile.

1.1 JWT Token Handling

Authentication Model

MEDDF assumes backend provides:

- JWT access token (short-lived)
- Optional refresh token (managed by backend)

Storage Strategy

Recommended:

- Store access token **in memory only** (Zustand volatile slice)
- Do NOT persist in localStorage
- Do NOT persist in sessionStorage (unless absolutely required)

Reason:

- XSS can exfiltrate localStorage
- Memory storage limits exposure window

Axios Interceptor Integration

```
apiClient.interceptors.request.use((config) => {
 const token = authStore.getState().accessToken
 if (token) {
   config.headers.Authorization = `Bearer ${token}`
```

```
 }
 return config
})
```

This ensures:

- Token injection centralized
- No component-level token handling
- No leakage into UI layer

Token Expiration Handling

Response interceptor:

```
apiClient.interceptors.response.use(
 response => response,
 error => {
   if (error.response?.status === 401) {
     triggerReauthentication()
   }
   return Promise.reject(error)
 }
)
```

Prevents silent failures.

1.2 XSS Prevention

Primary attack vectors:

- Configuration injection
- Dynamic telemetry values
- Third-party script injection

Mitigation Strategy

1. Never use `dangerouslySetInnerHTML`
2. Strict Content Security Policy (CSP)
3. Validate all configuration through schema
4. Render only primitive values
5. Sanitize dynamic strings if needed

Example CSP:

```
Content-Security-Policy:
default-src 'self';
connect-src 'self' wss://api.example.com;
script-src 'self';
```

1.3 Configuration Validation

Configuration is treated as untrusted input.

Zod validation layer ensures:

- Required keys present
- Types correct
- Position values valid
- Tile types allowed

VehicleConfigSchema.parse(config)

Invalid config halts rendering safely.

Prevents:

- Rendering undefined components
- Layout corruption

- Runtime crashes

1.4 API Endpoint Isolation

All network communication is routed through a single API client.

```
const apiClient = axios.create({
 baseURL: import.meta.env.VITE_API_BASE
})
```

Benefits:

- No hardcoded endpoints
- Centralized security headers
- Easy environment switching
- Reduced attack surface

---

2. Reliability Engineering

Reliability is designed to ensure:

- Dashboard remains usable during backend instability
- UI never fully crashes
- Network failures handled gracefully

2.1 Retry Logic

Applied to:

- WebSocket reconnection
- REST data fetch

Exponential Backoff:

```
const delay = Math.min(baseDelay * 2 ** attempt, maxDelay)
```

Prevents network storm.

2.2 Circuit Breaker Pattern

State Machine:

CLOSED → OPEN → HALF-OPEN → CLOSED

Logic:

- If repeated failures exceed threshold → OPEN
- Suspend requests
- After cooldown → HALF-OPEN
- If successful → CLOSED

Prevents cascading failures.

2.3 Polling Fallback

If WebSocket disconnects:

1. Switch to REST polling.
2. Attempt background reconnection.
3. Restore WebSocket when stable.

Pseudo:

```
if (socketClosed) {
 startPolling()
}
```

Ensures telemetry continuity.

2.4 Error Boundaries

React Error Boundary prevents full application crash.

```
class DashboardErrorBoundary extends React.Component {
 componentDidCatch(error, info) {
   logError(error)
 }
}
```

Wraps DashboardEngine.

If a tile fails:

- Only that subtree affected.
- UI remains operational.

---

3. Scalability Strategy

3.1 Multi-Vehicle Support

Architecture supports:

- Unlimited vehicle configurations
- Runtime vehicle switching

- URL-based context detection

Vehicle onboarding requires:

1. Add configuration entry.
2. Ensure telemetry keys match.
3. No engine modification.

Time complexity independent of vehicle count.

3.2 Plugin Tile System

TileRegistry enables plugin architecture.

TileRegistry.register("heatmap", HeatmapTile)

Engine depends on abstraction, not concrete tiles.

Benefits:

- Add new visualization types without refactoring.
- Open-Closed Principle compliance.
- Third-party extension ready.

3.3 Multi-Tenant Readiness

For SaaS EV platforms:

Store structure can include:

```
{
 tenantId,
 vehicleId,
 config,
```

```
 telemetry
}
```

Isolation via:

- Tenant-specific config namespace
- API namespace segmentation
- Route-based segregation

Supports:

- White-label dashboards
- Brand customization
- Tenant-specific tile sets

---

4. DevOps & Deployment Strategy

4.1 Environment Configuration

Managed via Vite:

```
VITE_API_BASE=https://api.production.com
VITE_WS_BASE=wss://ws.production.com
```

Environment injected at build time.

No runtime mutation.

4.2 Production Build

Vite production build performs:

- Tree shaking
- Code splitting
- Minification
- Asset hashing
- Dead code elimination

Output:

```
dist/
 index.html
 assets/*.js
 assets/*.css
```

Optimized for CDN delivery.

4.3 CI/CD Pipeline

Typical pipeline:

```
Git Push
↓
Lint & Type Check
↓
Unit Tests
↓
Build
↓
Docker Image Creation
↓
Deployment
```

Can integrate:

- GitHub Actions
- GitLab CI

- Jenkins

Ensures:

- Deterministic builds
- Automated testing
- Controlled release process

4.4 Versioning Strategy

Semantic Versioning:

MAJOR.MINOR.PATCH

- MAJOR → Breaking architecture changes
- MINOR → Feature additions
- PATCH → Bug fixes

Ensures compatibility tracking.

4.5 Docker Deployment Concept

Dockerfile:

```
FROM nginx:alpine
COPY dist /usr/share/nginx/html
```

Stateless container.

Advantages:

- Horizontal scalability

- Immutable deployment
- Easy rollback

Enterprise Risk Mitigation

MEDDF mitigates enterprise risks by:

- Centralized API abstraction
- Schema-validated configuration
- Reconnection resilience
- Error isolation
- Immutable state model
- Controlled dependency graph
- Containerized deployment readiness

14. MEDDF – Enterprise Readiness Specification

Audience: CTO / Principal Engineers / Platform Architects
Scope: Security, Reliability, Scalability, and DevOps Hardening

This section defines how MEDDF transitions from a high-performance frontend framework into a production-ready, enterprise-grade platform capable of operating securely and reliably in large-scale EV ecosystems.

---

1. Security Architecture

Security in MEDDF is based on **zero-trust frontend assumptions**.
All network boundaries are treated as hostile.

1.1 JWT Token Handling

Authentication Model

MEDDF assumes backend provides:

- JWT access token (short-lived)
- Optional refresh token (managed by backend)

Storage Strategy

Recommended:

- Store access token **in memory only** (Zustand volatile slice)
- Do NOT persist in localStorage
- Do NOT persist in sessionStorage (unless absolutely required)

Reason:

- XSS can exfiltrate localStorage
- Memory storage limits exposure window

Axios Interceptor Integration

```
apiClient.interceptors.request.use((config) => {
 const token = authStore.getState().accessToken
 if (token) {
   config.headers.Authorization = `Bearer ${token}`
 }
 return config
})
```

This ensures:

- Token injection centralized
- No component-level token handling
- No leakage into UI layer

Token Expiration Handling

Response interceptor:

```
apiClient.interceptors.response.use(
 response => response,
 error => {
   if (error.response?.status === 401) {
     triggerReauthentication()
   }
   return Promise.reject(error)
 }
)
```

Prevents silent failures.

1.2 XSS Prevention

Primary attack vectors:

- Configuration injection
- Dynamic telemetry values
- Third-party script injection

Mitigation Strategy

1. Never use `dangerouslySetInnerHTML`

2. Strict Content Security Policy (CSP)
3. Validate all configuration through schema
4. Render only primitive values
5. Sanitize dynamic strings if needed

Example CSP:

```
Content-Security-Policy:
default-src 'self';
connect-src 'self' wss://api.example.com;
script-src 'self';
```

1.3 Configuration Validation

Configuration is treated as untrusted input.

Zod validation layer ensures:

- Required keys present
- Types correct
- Position values valid
- Tile types allowed

```
VehicleConfigSchema.parse(config)
```

Invalid config halts rendering safely.

Prevents:

- Rendering undefined components
- Layout corruption
- Runtime crashes

1.4 API Endpoint Isolation

All network communication is routed through a single API client.

```
const apiClient = axios.create({
 baseURL: import.meta.env.VITE_API_BASE
})
```

Benefits:

- No hardcoded endpoints
- Centralized security headers
- Easy environment switching
- Reduced attack surface

---

2. Reliability Engineering

Reliability is designed to ensure:

- Dashboard remains usable during backend instability
- UI never fully crashes
- Network failures handled gracefully

2.1 Retry Logic

Applied to:

- WebSocket reconnection

- REST data fetch

Exponential Backoff:

const delay = Math.min(baseDelay * 2 ** attempt, maxDelay)

Prevents network storm.

2.2 Circuit Breaker Pattern

State Machine:

CLOSED → OPEN → HALF-OPEN → CLOSED

Logic:

- If repeated failures exceed threshold → OPEN
- Suspend requests
- After cooldown → HALF-OPEN
- If successful → CLOSED

Prevents cascading failures.

2.3 Polling Fallback

If WebSocket disconnects:

1. Switch to REST polling.
2. Attempt background reconnection.
3. Restore WebSocket when stable.

Pseudo:

```
if (socketClosed) {
 startPolling()
}
```

Ensures telemetry continuity.

2.4 Error Boundaries

React Error Boundary prevents full application crash.

```
class DashboardErrorBoundary extends React.Component {
 componentDidCatch(error, info) {
   logError(error)
 }
}
```

Wraps DashboardEngine.

If a tile fails:

- Only that subtree affected.
- UI remains operational.

---

3. Scalability Strategy

3.1 Multi-Vehicle Support

Architecture supports:

- Unlimited vehicle configurations
- Runtime vehicle switching
- URL-based context detection

Vehicle onboarding requires:

1. Add configuration entry.
2. Ensure telemetry keys match.
3. No engine modification.

Time complexity independent of vehicle count.

3.2 Plugin Tile System

TileRegistry enables plugin architecture.

TileRegistry.register("heatmap", HeatmapTile)

Engine depends on abstraction, not concrete tiles.

Benefits:

- Add new visualization types without refactoring.
- Open-Closed Principle compliance.
- Third-party extension ready.

3.3 Multi-Tenant Readiness

For SaaS EV platforms:

Store structure can include:

```
{
 tenantId,
 vehicleId,
 config,
 telemetry
}
```

Isolation via:

- Tenant-specific config namespace
- API namespace segmentation
- Route-based segregation

Supports:

- White-label dashboards
- Brand customization
- Tenant-specific tile sets

---

4. DevOps & Deployment Strategy

4.1 Environment Configuration

Managed via Vite:

```
VITE_API_BASE=https://api.production.com
VITE_WS_BASE=wss://ws.production.com
```

Environment injected at build time.

No runtime mutation.

4.2 Production Build

Vite production build performs:

- Tree shaking

- Code splitting
- Minification
- Asset hashing
- Dead code elimination

Output:

```
dist/
 index.html
 assets/*.js
 assets/*.css
```

Optimized for CDN delivery.

4.3 CI/CD Pipeline

Typical pipeline:

```
Git Push
↓
Lint & Type Check
↓
Unit Tests
↓
Build
↓
Docker Image Creation
↓
Deployment
```

Can integrate:

- GitHub Actions
- GitLab CI
- Jenkins

Ensures:

- Deterministic builds
- Automated testing
- Controlled release process

4.4 Versioning Strategy

Semantic Versioning:

MAJOR.MINOR.PATCH

- MAJOR → Breaking architecture changes
- MINOR → Feature additions
- PATCH → Bug fixes

Ensures compatibility tracking.

4.5 Docker Deployment Concept

Dockerfile:

```
FROM nginx:alpine
COPY dist /usr/share/nginx/html
```

Stateless container.

Advantages:

- Horizontal scalability
- Immutable deployment
- Easy rollback

Enterprise Risk Mitigation

MEDDF mitigates enterprise risks by:

- Centralized API abstraction
- Schema-validated configuration
- Reconnection resilience
- Error isolation
- Immutable state model
- Controlled dependency graph
- Containerized deployment readiness

15. MEDDF – Enterprise Architecture Audit Report

Auditor: Principal Software Architect (30+ years enterprise systems)
Scope: Full frontend architecture evaluation against enterprise-grade standards

This is a structural audit — not documentation review — but architectural compliance review.

---

1. Configuration-Driven Layout

✔ Is layout fully configuration-driven?

Current State: ~90% Compliant

You have:

- Grid parameters in config
- Tile positions in config
- Theme in config
- Tile types defined externally

Minor Gaps

- No formal versioning for config schema
- No runtime config version compatibility check
- Rule engine not fully integrated (optional but enterprise-level expected)

Required Refinements

- Add config version field
- Add backward compatibility handler
- Add dynamic feature flag support

2.Store Genericity

✔ Is store fully generic?

Current State: ~85% Compliant

You have:

- Telemetry slice
- UI slice
- Connection slice

However:

- Store structure still assumes telemetry structure
- No namespaced store for multi-tenant
- No plugin state extension mechanism

Required Refinements

- Introduce namespaced store pattern:

  state[vehicleId].telemetry

- Support dynamic store extension via plugins
- Add store middleware for logging / observability

3.Engine Free of Vehicle-Specific Logic

✔ Is engine vehicle-agnostic?

Current State: ~95% Compliant

Strengths:

- No conditional rendering per vehicle
- No vehicle-based switch-case
- Config-based rendering

Minor risk:

- Hardcoded assumptions about telemetry keys
- Potential hidden assumptions in tile components

Required Refinements

- Strict enforcement: tile must rely ONLY on props
- No fallback logic referencing specific telemetry names

---

4. Tile Plugin Readiness

✔ Are tiles plugin-ready?

Current State: ~80% Compliant

You have:

- TileRegistry mapping
- Factory resolution

Missing:

- Dynamic runtime registration API
- Plugin lifecycle hooks
- Plugin validation schema
- Plugin isolation boundary

Required Refinements

- Implement:

  registerTile(type, component, schema)

- Add tile capability metadata
- Add lazy plugin loading

5.Performance Optimization

✔ Is performance optimized?

Current State: ~85% Compliant

Strengths:

- Shallow immutable updates
- Selector-based subscription
- Memoized components
- Throttling strategy

Missing:

- Telemetry stress test benchmarking
- Automated frame drop detection
- Performance monitoring integration
- Profiling in production mode

Required Refinements

- Integrate performance instrumentation
- Add telemetry frequency guard
- Add FPS monitoring hook

6.Security Layer

✔ Is security implemented?

Current State: ~75% Compliant

You documented:

- JWT handling
- Axios interceptors
- Config validation
- CSP strategy

Missing:

- Token refresh lifecycle
- CSRF mitigation (if cookie-based auth)
- Config signature verification
- Secure WebSocket authentication

Required Refinements

- Implement refresh token flow
- Add signed configuration verification
- Secure WS auth headers
- Strict CSP enforcement

7.Error Handling

✔ Is error handling complete?

Current State: ~80% Compliant

You have:

- Error boundaries
- Retry logic
- Circuit breaker
- Polling fallback

Missing:

- Centralized logging
- Structured error reporting
- Telemetry health monitoring
- Crash recovery state

Required Refinements

- Add error logging abstraction layer
- Integrate monitoring (Sentry or equivalent)
- Add degraded-mode UI state

8.Scalability Strategy

✔ Is scalability realistic?

Current State: ~88% Compliant

Strengths:

- Multi-vehicle support
- Config-driven onboarding
- Plugin-ready registry
- Docker deployment

Weaknesses:

- No config lazy loading strategy
- No CDN config delivery model
- No micro-frontend federation example
- No large-fleet stress modeling

Required Refinements

- Load config per vehicle dynamically
- CDN-backed config storage
- Consider Module Federation compatibility
- Add stress modeling documentation